

RAPPORT TECHNIQUE COMPLET

e-Store

Application Web E-Commerce Full-Stack
Architecture, Implémentation et Analyse Approfondie

PROJET RÉALISÉ PAR
Walid Ait Sidi Ahmed
Chaimaa El Mehdaoui

PROJET ENCADRÉ PAR
Omar Zahour
Encadrant

Année Universitaire 2025 – 2026

Contents

1	Introduction et Présentation du Projet	4
1.1	Objectifs du Projet	4
1.2	Périmètre Fonctionnel	4
1.3	Contraintes Techniques	5
2	Modélisation UML	6
2.1	Diagramme de Cas d'Utilisation	6
2.2	Diagramme de Classes	7
2.3	Diagramme de Séquence	7
3	Architecture Globale	9
3.1	Vue d'Ensemble	9
3.2	Architecture Backend — Pattern en Couches	9
3.3	Architecture Frontend — Organisation par Fonctionnalité	10
3.4	Modules de Domaine Backend	11
4	Conception de la Base de Données	12
4.1	Schéma des Tables	12
4.2	Relations JPA Détaillées	13
4.2.1	Relations @OneToOne	13
4.2.2	Relations @OneToMany / @ManyToOne	13
4.2.3	Annotation @PrePersist	14
4.3	Données de Démarrage (DataInitializer)	14
4.3.1	Produits et Stocks Initiaux	14
5	Module Customer — Gestion des Utilisateurs	15
5.1	Entités User et Profile	15
5.2	CustomerService — Logique Métier	15
5.3	Endpoints API — /api/auth	16
6	Module Catalog — Catalogue Produits	17
6.1	Entités Product, Category, Review	17
6.2	CatalogService — Agrégation Multi-Source	17
6.2.1	Recherche Textuelle — Requête JPQL Personnalisée	17
6.3	Endpoints API — /api/products et /api/categories	18
7	Module Inventory — Gestion des Stocks	19
7.1	Entité Inventory	19
7.2	InventoryService — Méthodes Clés	19
8	Module Shopping — Gestion du Panier	20
8.1	Entités Cart et CartItem	20
8.2	ShoppingService — Logique Métier	20
8.3	Endpoints API — /api/cart	21

9	Module Billing — Gestion des Commandes	22
9.1	Entités Order et OrderItem	22
9.2	Flux de Création d'une Commande — @Transactional	23
9.3	Endpoints API — /api/orders	24
10	Application Frontend React	25
10.1	Routing — App.js	25
10.2	Couche Service — api.js (Axios)	26
10.3	Contextes React	26
11	Composants Frontend — Détail Technique	27
11.1	Header et Navigation	27
11.2	ProductList — Catalogue et Filtres	27
11.3	ProductCard — Carte Produit	27
11.4	ProductDetail — Fiche Produit	28
11.5	Cart — Panier et Checkout	28
11.6	OrderHistory et OrderConfirmation	28
12	Gestion Centralisée des Exceptions	29
12.1	Structure de la Réponse d'Erreur	29
12.2	Exceptions Gérées et Codes HTTP	29
13	Configuration et Infrastructure	31
13.1	application.properties — Backend	31
13.2	Configuration CORS — CorsConfig.java	31
13.3	Dépendances Maven — pom.xml	32
14	Flux de Données End-to-End	33
14.1	Flux d'Inscription et de Connexion	33
14.2	Flux d'Ajout au Panier	33
14.3	Flux de Passage en Caisse	34
15	Analyse de Sécurité	35
15.1	Vulnérabilités Critiques — Correction Requise en Production	35
15.2	Risques Modérés et Points Conformes	36
16	Qualité du Code et Bonnes Pratiques	37
16.1	Lombok — Réduction du Boilerplate	37
16.2	Transactions Spring @Transactional	37
16.3	Pattern DTO — Découplage Entité/API	37
16.4	BigDecimal pour les Valeurs Monétaires	38
16.5	Absence de Tests Automatisés	38
17	Expérience Utilisateur et Interface	39
17.1	Positionnement Design	39
17.2	États de Chargement	39
17.3	États Vides	39
17.4	Accessibilité	39
18	Feuille de Route et Améliorations	40
18.1	Priorité Haute — Sécurité (Pré-Production Obligatoire)	40
18.2	Priorité Haute — Fonctionnalités Manquantes	40
18.3	Priorité Moyenne — Améliorations Fonctionnelles	40
18.4	Priorité Basse — Mise en Production	41

19 Guide de Démarrage	42
19.1 Prérequis	42
19.2 Démarrage du Backend	42
19.3 Démarrage du Frontend	43
19.4 Vérification du Bon Fonctionnement	43
20 Glossaire Technique	44
21 Conclusion	46

1 Introduction et Présentation du Projet

01

Introduction

Objectifs, périmètre et contraintes

eStore est une application web e-commerce complète développée en full-stack, combinant un backend Java Spring Boot avec un frontend React 18. Ce projet constitue une démonstration académique et technique d'une boutique en ligne fonctionnelle, couvrant l'intégralité du parcours client : depuis la navigation dans le catalogue produits jusqu'à la confirmation de commande, en passant par la gestion du panier, l'authentification utilisateur, la liste de souhaits et l'historique des commandes.

L'application suit une architecture en trois tiers bien définie : la couche de présentation (React SPA), la couche métier (Spring Boot REST API), et la couche de données (MySQL 8). Cette séparation stricte des responsabilités garantit la maintenabilité, la testabilité et l'évolutivité du code. Chaque module est organisé selon le principe de responsabilité unique, avec un découpage par domaine métier plutôt que par couche technique.

1.1 Objectifs du Projet

Le projet vise plusieurs objectifs pédagogiques et techniques précis :

- Démontrer une architecture backend Spring Boot organisée en modules de domaine (customer, catalog, inventory, shopping, billing) avec une séparation stricte Controller → Service → Repository
- Implémenter le flux d'achat complet de bout en bout : recherche produit → fiche détail → panier → commande transactionnelle → confirmation
- Assurer une gestion d'inventaire en temps réel avec vérification des stocks lors de chaque ajout au panier et lors du passage en caisse
- Mettre en œuvre la gestion de l'état côté frontend avec React Context API : AuthContext, CartContext, WishlistContext
- Exposer une API REST cohérente, sécurisée par CORS, consommée par le frontend via Axios avec une couche de service abstraite
- Illustrer les bonnes pratiques : DTOs distincts des entités, services transactionnels, gestion centralisée des exceptions, Bean Validation

1.2 Périmètre Fonctionnel

eStore couvre les fonctionnalités suivantes, toutes implémentées et opérationnelles :

Fonctionnalité	Description Détaillée
Catalogue Produits	Navigation, recherche textuelle insensible à la casse sur nom et description, filtrage par catégorie, tri par prix et par note
Fiche Produit	Image, description, prix, note moyenne calculée, nombre d'avis, stock disponible en temps réel, sélecteur de quantité borné par le stock
Inscription / Connexion	Formulaire d'inscription avec validation, création automatique de profil ; authentification email/mot de passe, session local-Storage
Panier d'Achat	Ajout avec vérification stock, modification de quantité, suppression d'article, vidange complète, total calculé avec sous-totaux
Validation Inventaire	Vérification du stock à chaque opération de panier et lors du passage en caisse — erreur explicite si rupture
Passage en Caisse	Création de commande atomique @Transactional, réservation de stock, nettoyage automatique du panier
Confirmation Commande	Page récapitulative avec numéro de commande, articles commandés, montant total, statut et informations client
Historique Commandes	Liste chronologique triée par date décroissante, avec détail des articles et badges de statut colorés
Liste de Souhaits	Ajout/retrait depuis la fiche ou la carte, badge compteur dans le Header, persistance localStorage
Avis et Notes	Entités backend complètes avec calcul de note moyenne via requête JPQL agrégée

1.3 Contraintes Techniques

- Aucune dépendance à des frameworks CSS externes (Bootstrap, Tailwind, Material UI) — tous les styles sont développés en CSS pur
- Gestion d'état exclusivement via React Context API, sans bibliothèque tierce (Redux, Zustand)
- Persistance de session client via localStorage (pas de cookies HttpOnly ni de tokens JWT dans la version actuelle)
- Déploiement en environnement de développement local uniquement — pas de configuration cloud ni Docker

2 Modélisation UML

02

Modélisation UML

Diagrammes de cas d'utilisation, de classes et de séquence

La modélisation UML du projet e-Store fournit une représentation structurée et visuelle de l'ensemble du système. Ces diagrammes constituent la documentation de conception qui guide l'implémentation et facilite la compréhension du système par toutes les parties prenantes.

2.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation permet de représenter les interactions entre les différents acteurs du système et les principales fonctionnalités offertes par l'application e-Store.

Les acteurs principaux identifiés sont le **Client** et le système **E-Store**. Le client peut consulter le catalogue, gérer son panier, passer des commandes, s'inscrire et se connecter, laisser des avis, consulter son historique et se déconnecter. Ce diagramme met en évidence l'ensemble du parcours utilisateur de la boutique en ligne, depuis l'authentification jusqu'à la validation finale de commande.

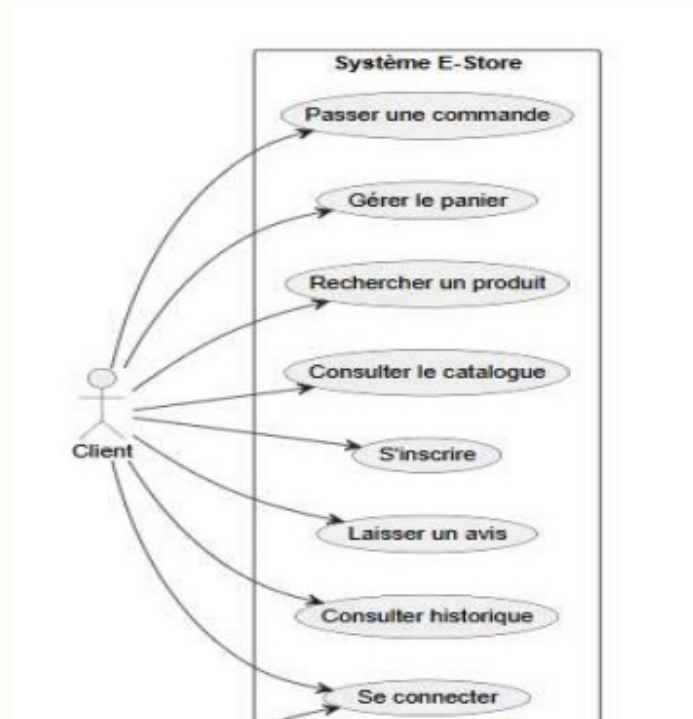


Figure 2.1: Diagramme de cas d'utilisation — acteurs et fonctionnalités du système e-Store

2.2 Diagramme de Classes

Le diagramme de classes représente la structure générale du système ainsi que les différentes entités utilisées dans l'application e-Store.

Il met en évidence les principales classes telles que `User`, `Product`, `Category`, `Cart`, `Order`, `Inventory` et `Review`, ainsi que leurs relations et multiplicités au sein de la base de données. On y observe notamment les associations `@OneToOne` entre `User` et `Profile`, les associations `@OneToMany` entre `Cart` et `CartItem`, ainsi que les dépendances entre `Product`, `Category` et `Inventory`. La classe `Review`, annotée «MongoDB» dans le diagramme, représente les avis produits avec leurs attributs de notation.

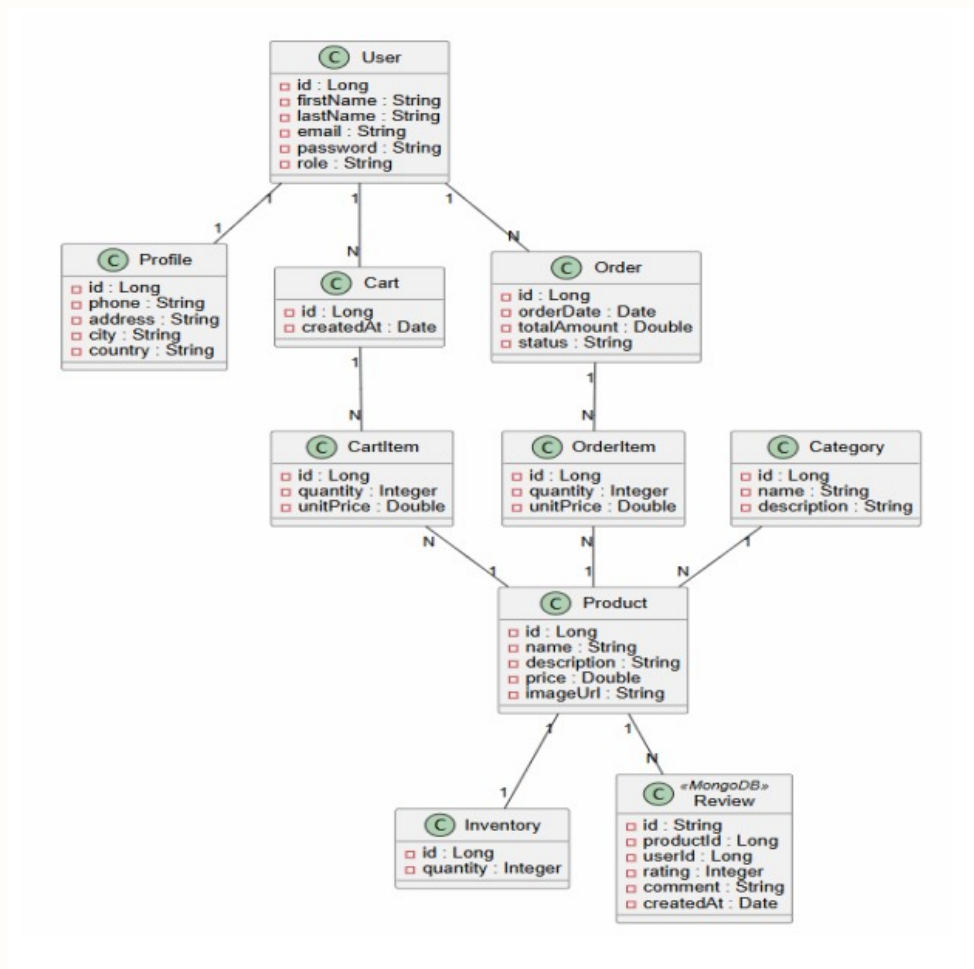


Figure 2.2: Diagramme de classes — entités JPA, attributs et relations du système e-Store

2.3 Diagramme de Séquence

Les diagrammes de séquence permettent de représenter le déroulement chronologique des échanges entre les composants du système lors de l'exécution d'un scénario fonctionnel.

Dans le cadre du projet e-Store, le scénario de **validation d'une commande** a été modélisé en détail. Ce diagramme illustre les interactions entre le **Client**, le frontend (**Angular/React**), le **Spring Boot** backend et la couche **JPA/MySQL**. On y distingue clairement : le déclenchement par le client du clic sur "Valider commande", l'émission d'un `POST /api/orders`, la vérification du stock côté serveur, la création transactionnelle de

l'Order et des OrderItems, la mise à jour de l'inventaire, le vidage du panier, et enfin le retour d'un 201 Created avec l'affichage de la page de confirmation.

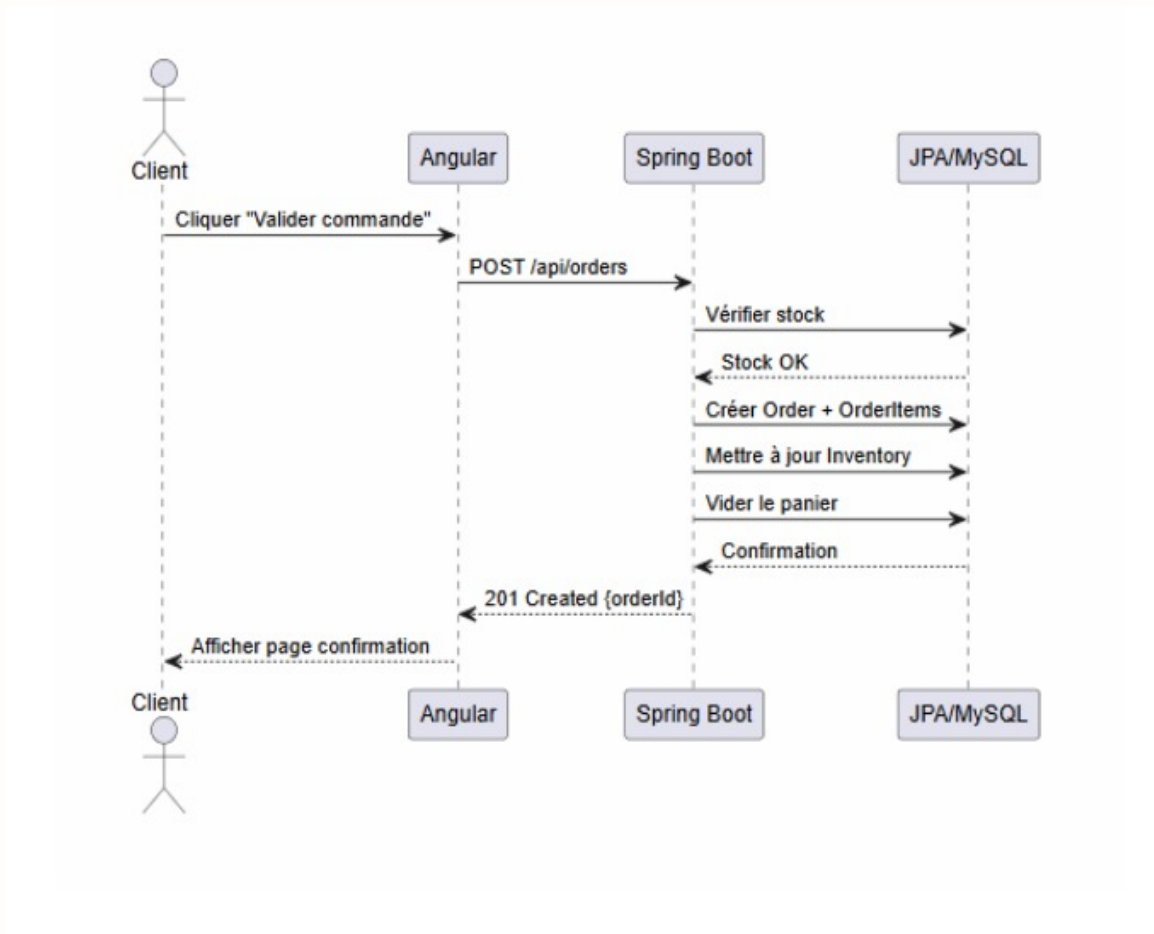


Figure 2.3: Diagramme de séquence — flux de validation d'une commande (checkout transactionnel)

3 Architecture Globale

03

Architecture

Trois tiers, modules de domaine et organisation du code

L'architecture d'eStore repose sur le modèle client-serveur classique en trois tiers, avec une séparation nette entre présentation, logique métier et accès aux données. Cette approche favorise la modularité, facilite les tests unitaires de chaque couche isolément, et permet une évolution indépendante du frontend et du backend.

3.1 Vue d'Ensemble

Le navigateur client héberge une Single Page Application React qui communique exclusivement avec l'API Spring Boot via des requêtes HTTP REST. Le serveur Spring Boot, déployé sur le port 8080, traite les requêtes, applique la logique métier et interagit avec la base de données MySQL via Spring Data JPA / Hibernate. La configuration CORS autorise les requêtes du serveur de développement React sur le port 3000.

Note d'Architecture

La communication entre le frontend et le backend est entièrement asynchrone et basée sur JSON (`application/json`). Aucun rendu côté serveur (SSR) n'est utilisé : toute la logique d'affichage réside dans le bundle JavaScript React. Chaque requête HTTP est stateless du point de vue du serveur.



Figure 3.1: Architecture trois tiers d'eStore — séparation stricte des responsabilités

3.2 Architecture Backend — Pattern en Couches

Le backend suit rigoureusement le pattern MVC adapté aux APIs REST. Chaque module de domaine contient cinq sous-couches :

Couche	Annotation Spring	Responsabilité
Controller	@RestController	Définit les endpoints REST, gère les codes HTTP, valide les entrées avec @Valid, délègue au service
Service	@Service	Contient toute la logique métier, les validations fonctionnelles, l'orchestration entre domaines, les transactions
Repository	@Repository	Interface Spring Data JPA pour les opérations CRUD et les requêtes JPQL personnalisées
Entity	@Entity	Classes Java mappées sur les tables MySQL via annotations JPA (@Table, @Column, @OneToMany...)
DTO	POJO	Objets de transfert : Request (entrée validée) et Response (sortie sélectionnée, sans champs sensibles)



Figure 3.2: Pattern en couches du backend Spring Boot — flux unidirectionnel Controller → Service → Repository

3.3 Architecture Frontend — Organisation par Fonctionnalité

- `src/services/api.js` — Couche d'abstraction HTTP : toutes les fonctions appelant l'API via Axios
- `src/contexts/` — Fournisseurs de contexte : `AuthContext`, `CartContext`, `WishlistContext`
- `src/components/` — Composants par domaine : `auth/`, `catalog/`, `cart/`, `orders/`, `common/`, `wishlist/`
- `src/App.js` — `BrowserRouter` React Router DOM 6 avec protection des routes privées via `ProtectedRoute`

3.4 Modules de Domaine Backend

Module de Domaine	Responsabilité Principale
<code>com.estore.customer</code>	Inscription avec création de profil automatique, authentification par email/mot de passe, consultation de profil
<code>com.estore.catalog</code>	Produits, catégories, recherche textuelle JPQL, agrégation de notes et avis
<code>com.estore.inventory</code>	Consultation du niveau de stock, vérification de disponibilité, réservation transactionnelle
<code>com.estore.shopping</code>	Panier get-or-create, ajout/modification/suppression d'articles, vérification stock à chaque opération
<code>com.estore.billing</code>	Création atomique <code>@Transactional</code> , orchestration inventory+cart, historique par utilisateur
<code>com.estore.config</code>	CORS (<code>CorsConfig</code>), ressources statiques (<code>WebConfig</code>), données de démarrage (<code>DataInitializer</code>)
<code>com.estore.exception</code>	<code>GlobalExceptionHandler</code> (<code>@RestControllerAdvice</code>), exceptions métier personnalisées

4 Conception de la Base de Données

04

Base de Données

Entités JPA, relations et données initiales

La base de données eStore est composée de neuf tables relationnelles, générées automatiquement par Hibernate grâce à la propriété `spring.jpa.hibernate.ddl-auto=update`. Cette configuration permet à Hibernate de créer les tables manquantes et d'ajouter les colonnes absentes au démarrage, sans jamais supprimer de données existantes.

4.1 Schéma des Tables

Table	Clé Primaire	Colonnes et Contraintes
users	id BIGINT AUTO_INC	firstName (NOT NULL), lastName (NOT NULL), email (UNIQUE NOT NULL), password (NOT NULL), createdAt (@PrePersist)
profiles	id BIGINT AUTO_INC	user_id (FK UNIQUE), phone, address, city, country, postalCode — OneToOne avec users
categories	id BIGINT AUTO_INC	name (UNIQUE NOT NULL), description TEXT
products	id BIGINT AUTO_INC	name (NOT NULL), description (TEXT), price (DECIMAL 10,2), imageUrl, category_id (FK)
inventories	id BIGINT AUTO_INC	product_id (FK UNIQUE), quantity (INT NOT NULL DEFAULT 0)
carts	id BIGINT AUTO_INC	user_id (FK UNIQUE) — un seul panier par utilisateur
cart_items	id BIGINT AUTO_INC	cart_id (FK), product_id (FK), quantity (INT), unitPrice (DECIMAL 10,2)
orders	id BIGINT AUTO_INC	user_id (FK), orderDate (@PrePersist), totalAmount (DECIMAL 10,2), status (VARCHAR)
order_items	id BIGINT AUTO_INC	order_id (FK), product_id (FK), quantity, unitPrice, subtotal (DECIMAL 10,2)
reviews	id BIGINT AUTO_INC	product_id (FK), user_id (FK), rating (INT 1-5), comment (VARCHAR 1000), createdAt

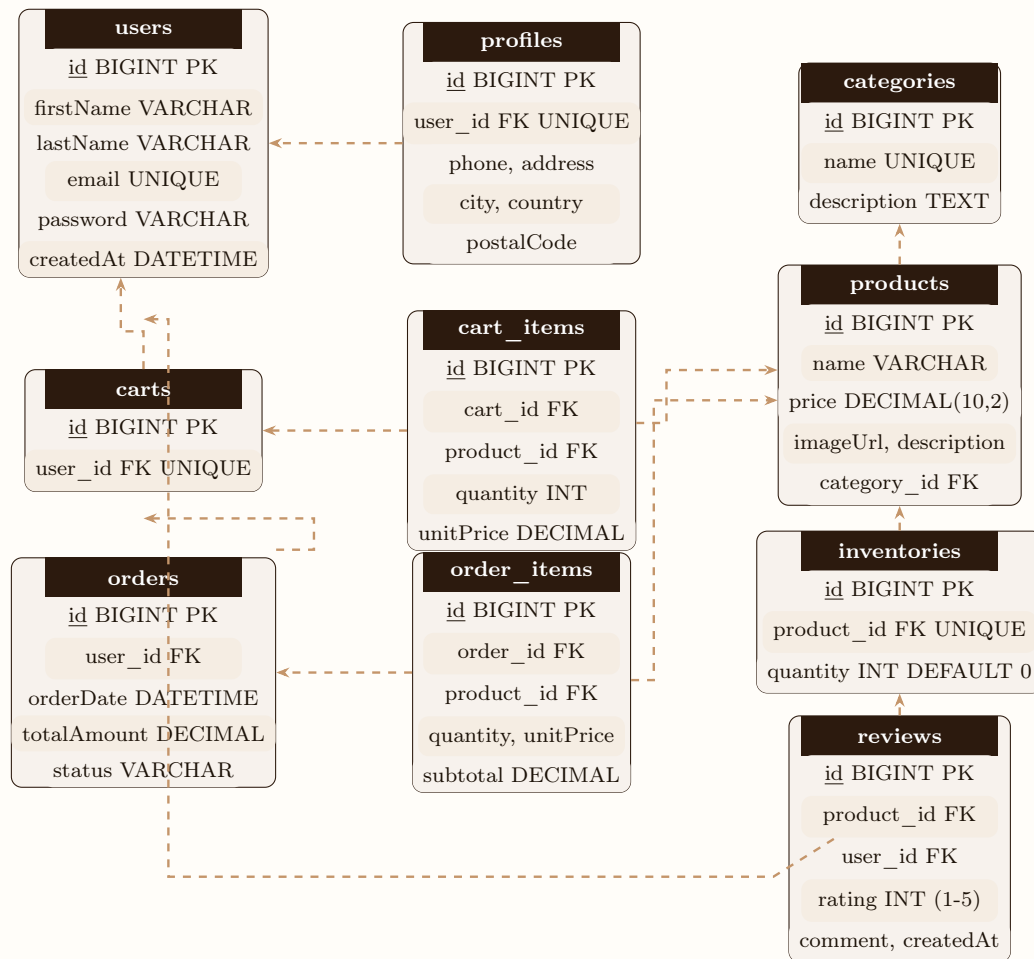


Figure 4.1: Schéma relationnel de la base de données eStore — 9 tables, clés étrangères et contraintes JPA

4.2 Relations JPA Détaillées

4.2.1 Relations @OneToOne

Deux relations OneToOne existent dans le schéma. La relation **User** → **Profile** (informations complémentaires) et la relation **User** → **Cart** (un panier par utilisateur). Ces deux relations utilisent `@JoinColumn` avec contrainte `unique=true`, garantissant l'unicité à la fois au niveau JPA et MySQL. Les profils sont créés vides automatiquement lors de l'inscription.

4.2.2 Relations @OneToMany / @ManyToOne

La relation la plus importante est **Order** → **OrderItem**, définie avec :

```

@OneToMany(mappedBy = "order",
            cascade = CascadeType.ALL,
            orphanRemoval = true)
private List<OrderItem> items;
    
```

Cette configuration garantit que supprimer une commande supprime automatiquement tous ses articles (cascade), et qu'un article retiré d'une commande est automatiquement supprimé de la BDD (orphanRemoval). La même configuration s'applique à **Cart** → **CartItem**.

4.2.3 Annotation @PrePersist

L'annotation `@PrePersist` sur une méthode `void` de l'entité déclenche son exécution juste avant la première insertion. `User` l'utilise pour initialiser `createdAt`, `Order` pour initialiser `orderDate` et `status=PENDING`, `Review` pour initialiser `createdAt`.

4.3 Données de Démarrage (DataInitializer)

`DataInitializer` implémente `CommandLineRunner` et est annotée `@Component`. Spring Boot exécute sa méthode `run()` au démarrage après l'initialisation du contexte. Elle vérifie `categoryRepository.count() == 0` avant toute insertion pour éviter les doublons.

4.3.1 Produits et Stocks Initiaux

Produit	Catégorie / Prix / Stock Initial
Laptop Pro	Electronics — 1 299,99 € — 10 unités
Smartphone X	Electronics — 899,99 € — 15 unités
Cotton T-Shirt	Clothing — 19,99 € — 50 unités
Mystery Novel	Books — 14,99 € — 30 unités

CRITIQUE — Mots de Passe en Clair

Les mots de passe des utilisateurs de test sont stockés **en clair** dans `DataInitializer` et en base de données. En production, il est impératif d'utiliser `BCryptPasswordEncoder` pour hasher tous les mots de passe avant persistance. Ce point est explicitement marqué
`// TODO: encode password dans CustomerService.`

5 Module Customer — Gestion des Utilisateurs

05

Module Customer

Authentification, inscription et profils

Le module `customer` est responsable de toute la gestion du cycle de vie des utilisateurs : inscription, authentification et consultation de profil. Il est exposé via `CustomerController` sur l'endpoint de base `/api/auth`.

5.1 Entités User et Profile

User

L'entité `User` contient : `id` (Long, `@GeneratedValue IDENTITY`), `firstName`, `lastName`, `email` (`@Column unique=true`), `password` (en clair dans la version actuelle), `createdAt` (initialisé via `@PrePersist`). L'unicité de l'email est garantie à deux niveaux : contrainte MySQL UNIQUE et vérification via `userRepository.existsByEmail()` dans le service.

Profile

L'entité `Profile` est liée à `User` par `@OneToOne`. Elle stocke des informations optionnelles : `phone`, `address`, `city`, `country`, `postalCode`. Un profil vide est automatiquement créé lors de l'inscription.

5.2 CustomerService — Logique Métier

`register(RegisterRequest)` — `@Transactional`

1. Vérifie l'unicité de l'email : lève `ResourceAlreadyExistsException` si doublon
2. Crée et persiste l'entité `User`
3. Crée et persiste un `Profile` vide associé
4. Retourne un `UserResponse` DTO (sans le champ `password`)

`login(LoginRequest)`

Récupère l'utilisateur par email (`ResourceNotFoundException` si introuvable), compare les mots de passe en clair. Placeholder pour une future implémentation BCrypt : `passwordEncoder.matches(request.getPassword(), user.getPassword())`.

5.3 Endpoints API — /api/auth

Méthode	Endpoint	Auth	Description
POST	/api/auth/register	Non	Inscription — body: {firstName, lastName, email, password} — 201 Created
POST	/api/auth/login	Non	Connexion — body: {email, password} — 200 OK avec UserResponse
GET	/api/auth/users/{id}	Non	Récupère les infos d'un utilisateur par ID

6 Module Catalog — Catalogue Produits

06

Module Catalog

Produits, catégories, recherche et avis

Le module `catalog` gère le catalogue produits : entités `Product`, `Category` et `Review`, opérations de lecture, recherche textuelle et calcul de notes. Ce module est en lecture seule depuis l'API publique.

6.1 Entités `Product`, `Category`, `Review`

Category

Contient `id`, `name` (`@Column unique=true`), `description`. Relation `@OneToMany(cascade=ALL)` vers `Product`.

Product

Contient `id`, `name`, `description` (TEXT), `price` (BigDecimal, precision=10 scale=2), `imageUrl`, et une relation `@ManyToOne` vers `Category`. Les images sont servies en statique depuis `/static/images/products/`.

Review

Contient `id`, relation `@ManyToOne` vers `Product` et vers `User`, `rating` (Integer 1–5), `comment` (VARCHAR 1000), `createdAt` (`@PrePersist`).

6.2 `CatalogService` — Agrégation Multi-Source

La méthode privée `mapToProductResponse(Product)` construit un `ProductResponse` enrichi depuis trois sources :

- Champs de base depuis l'entité : `id`, `name`, `description`, `price`, `imageUrl`
- Informations de catégorie : `categoryId`, `categoryName`
- Stock disponible depuis `inventoryRepository.findById()`
- Note moyenne depuis `reviewService.getProductAverageRating()`
- Nombre d'avis depuis `reviewRepository.getReviewCountByProductId()`

6.2.1 Recherche Textuelle — Requête JPQL Personnalisée

```
@Query("SELECT p FROM Product p WHERE " +  
        "LOWER(p.name) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +  
        "LOWER(p.description) LIKE LOWER(CONCAT('%', :keyword, '%'))")  
List<Product> searchByKeyword(@Param("keyword") String keyword);
```

Recherche insensible à la casse sur le nom ET la description. Les paramètres nommés

(:keyword) préviennent l'injection SQL via des PreparedStatements JDBC.

6.3 Endpoints API — /api/products et /api/categories

Méthode	Endpoint	Auth	Description
GET	/api/products	Non	Liste tous les produits avec stock et notes
GET	/api/products/{id}	Non	Produit par ID — 404 si introuvable
GET	/api/products/search?keyword=	Non	Recherche textuelle sur nom et description
GET	/api/categories	Non	Liste toutes les catégories
GET	/api/categories/{id}/products	Non	Produits d'une catégorie
GET	/api/reviews/product/{id}	Non	Avis d'un produit
POST	/api/reviews?userId=	userId param	Soumettre un avis

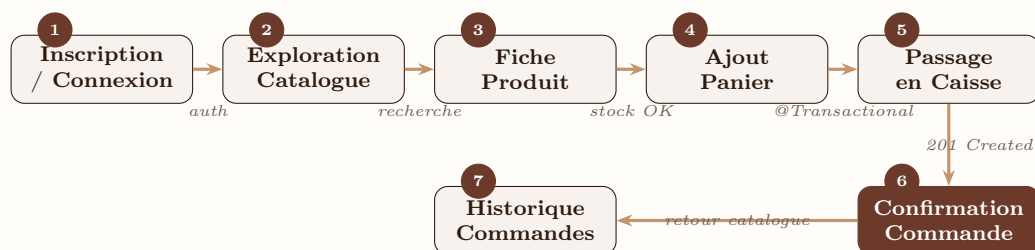


Figure 6.1: Parcours client complet — de l'inscription à la confirmation de commande

7 Module Inventory — Gestion des Stocks

07

Module Inventory

Stocks, disponibilité et réservation

Le module `inventory` est un module de support interne sans contrôleur REST public. Il est utilisé exclusivement par les modules `shopping` et `billing` pour vérifier et modifier les niveaux de stock, garantissant une encapsulation complète de la logique de gestion des stocks.

7.1 Entité Inventory

`Inventory` est liée à `Product` par `@OneToOne` avec `@JoinColumn(unique=true)`. Elle contient uniquement `quantity` (`Integer`, `nullable=false`, défaut 0). Un enregistrement est créé pour chaque produit par `DataInitializer`.

7.2 InventoryService — Méthodes Clés

`checkAvailability(productId, requestedQuantity)`

Retourne `boolean` : `true` si `inventory.quantity >= requestedQuantity`, `false` sinon (y compris si aucun enregistrement n'existe). Appelée en amont de toute opération de panier ou commande.

`reserveStock(productId, quantity)` — `@Transactional`

Méthode critique : récupère l'inventaire, double vérification du stock (pour gérer les conditions de course concurrentes), décrémente `quantity` et sauvegarde. Lève `RuntimeException("Insufficient stock")` si le stock est insuffisant lors de la vérification finale.

`updateStock(productId, newQuantity)`

Méthode d'administration : get-or-create de l'enregistrement `Inventory`, puis mise à jour de la quantité. Destinée à une future interface d'administration.

Pattern de Cohésion Forte

Les modules `shopping` et `billing` ne manipulent jamais directement `InventoryRepository`. Ils passent systématiquement par `InventoryService`, assurant une encapsulation complète et facilitant les modifications futures (alertes de rupture, réapprovisionnement automatique, etc.).

8 Module Shopping — Gestion du Panier

08

Module Shopping

Panier d'achat, articles et persistance serveur

Le module `shopping` gère l'ensemble du cycle de vie du panier. La persistance côté serveur (MySQL) est un choix architectural clé : le panier est sauvegardé entre les sessions et les appareils, et les règles métier (vérification de stock) sont appliquées côté serveur.

8.1 Entités `Cart` et `CartItem`

`Cart`

Liée à `User` par `@OneToOne`. Possède une collection via `@OneToMany(cascade=ALL, orphanRemoval=true)` vers `CartItem`. L'`orphanRemoval` garantit qu'un article retiré du `Set<CartItem>` est automatiquement supprimé de la BDD.

`CartItem`

Contient : `id`, référence `@ManyToOne` vers `Cart`, référence `@ManyToOne` vers `Product`, `quantity` (`Integer`), `unitPrice` (`BigDecimal` — capturé au moment de l'ajout, protège contre les variations de prix).

8.2 `ShoppingService` — Logique Métier

`getOrCreateCart(userId)` — Pattern Get-or-Create

Tente de récupérer le panier via `cartRepository.findById()`. Si aucun panier n'existe (première interaction), en crée et persiste un nouveau automatiquement. Transparent pour les couches appelantes.

`addToCart(userId, productId, quantity)`

1. `getOrCreateCart(userId)`
2. `inventoryService.checkAvailability()` — exception si stock insuffisant
3. Produit déjà dans le panier ? Oui → incrémentation + re-vérification du total ; Non → nouveau `CartItem` avec `unitPrice` courant
4. `cartRepository.save()` et retour de `CartResponse`

`mapToCartResponse(Cart)`

Calcule les sous-totaux (`unitPrice × quantity`) par article et le `totalAmount` global. Retourne `CartResponse` avec `totalItems` (nombre de lignes distinctes).

8.3 Endpoints API — /api/cart

Méthode	Endpoint	Auth	Description
GET	/api/cart/{userId}	Non	Récupère ou crée le panier de l'utilisateur
POST	/api/cart/{userId}/add	Non	Ajoute un produit — body: {productId, quantity}
PUT	/api/cart/{userId}/items/{id}?qty=	Non	Met à jour la quantité (0 = suppression)
DELETE	/api/cart/{userId}/items/{id}	Non	Supprime un article spécifique
DELETE	/api/cart/{userId}/clear	Non	Vide entièrement le panier — 204 No Content

9 Module Billing — Gestion des Commandes

09

Module Billing

Commandes et checkout transactionnel atomique

Le module `billing` est le plus complexe car il orchestre plusieurs domaines dans une transaction atomique. La création d'une commande implique la lecture du panier, la vérification des stocks, la création de l'ordre, la réservation du stock et le nettoyage du panier — le tout garanti par une seule annotation `@Transactional`.

9.1 Entités `Order` et `OrderItem`

`Order`

Contient : `id`, `user` (`@ManyToOne`), `orderDate` (`@PrePersist`), `totalAmount` (`BigDecimal`, `precision=10 scale=2`), `status` (`PENDING` / `CONFIRMED` / `SHIPPED` / `DELIVERED` / `CANCELLED`), `items` (`@OneToMany`, `cascade=ALL`, `orphanRemoval=true`).

`OrderItem`

Contient : `id`, `order` (`@ManyToOne`), `product` (`@ManyToOne`), `quantity`, `unitPrice` (snapshot immuable), `subtotal` (calculé lors de la création).

9.2 Flux de Création d'une Commande — @Transactional

#	Action	Détail	Exception si Échec
1	Récupération user	<code>findById(userId).orElseThrow()</code>	<code>ResourceNotFoundException</code>
2	Récupération panier	<code>findById(userId).orElseThrow()</code>	<code>RuntimeException</code>
3	Vérif. non vide	<code>items.isEmpty()</code> → erreur	<code>RuntimeException</code>
4	Vérif. stock global	<code>checkAvailability()</code> pour chaque <code>CartItem</code>	<code>RuntimeException</code> (nom produit)
5	Création Order	<code>new Order(), statut = CONFIRMED</code>	—
6	Création OrderItems	Pour chaque <code>CartItem</code> → <code>OrderItem + subtotal</code>	—
7	Calcul totalAmount	Somme des subtotals	—
8	Réservation stock	<code>reserveStock()</code> pour chaque article	<code>RuntimeException</code>
9	Persistance	<code>orderRepository.save(order)</code> avec cascade	—
10	Nettoyage panier	<code>shoppingService.clearCart(userId)</code>	—
11	Retour réponse	<code>mapToOrderResponse(savedOrder)</code>	—

Garantie Transactionnelle @Transactional

Si une exception est levée à n'importe quelle étape (stock insuffisant, produit introuvable...), Spring effectue un **rollback complet** de toutes les opérations précédentes. Il est impossible de créer une commande partielle ou de débiter le stock sans créer la commande correspondante. Cette garantie ACID est fondamentale pour l'intégrité des données e-commerce.

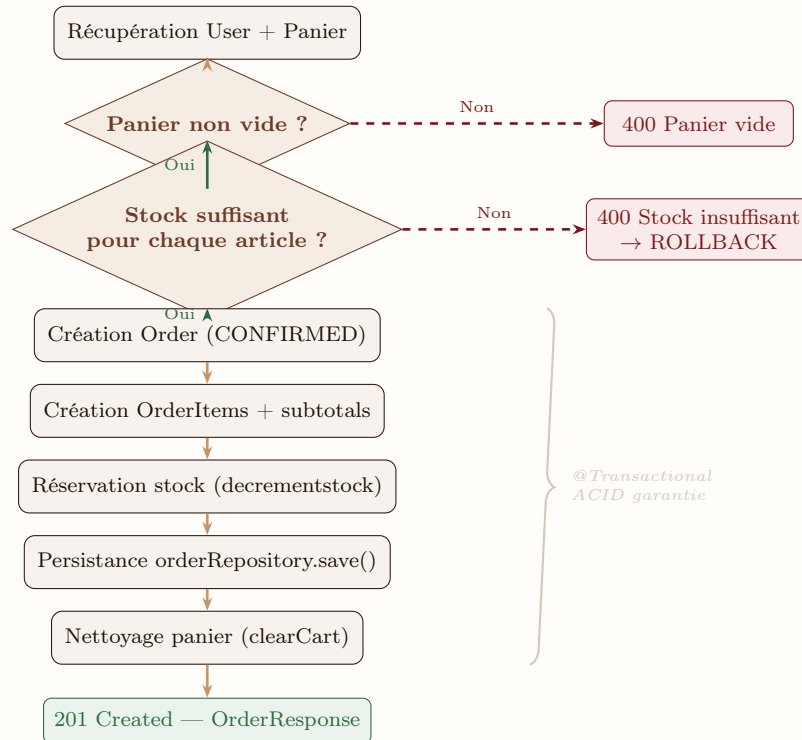


Figure 9.1: Flux transactionnel de création de commande — logique @Transactional avec rollback automatique

9.3 Endpoints API — /api/orders

Méthode	Endpoint	Auth	Description
POST	/api/orders?userId=	Non	Crée une commande — 201 Created avec OrderResponse complet
GET	/api/orders/user/{userId}	Non	Historique trié par date décroissante
GET	/api/orders/{orderId}	Non	Détail d'une commande avec tous ses articles

10 Application Frontend React

10

Frontend React

Routing, contextes et couche service Axios

L'application frontend est une Single Page Application React 18, créée avec Create React App. Elle utilise React Router DOM 6 pour la navigation, Axios pour les appels HTTP et le Context API natif pour la gestion d'état global. Aucune bibliothèque UI externe n'est utilisée.

10.1 Routing — App.js

App.js configure le BrowserRouter et déclare toutes les routes. Le composant ProtectedRoute protège les routes privées : il lit `useAuth()` pour l'état de connexion, affiche un spinner pendant le chargement (lecture du localStorage au montage), et redirige vers `/login` via `<Navigate>` si l'utilisateur n'est pas connecté.

Route	Composant / Protection
<code>/</code> et <code>/products</code>	ProductList — Publique
<code>/products/:id</code>	ProductDetail — Publique
<code>/login</code>	Login — Publique (redirige vers <code>/</code> si déjà connecté)
<code>/register</code>	Register — Publique
<code>/cart</code>	Cart — Protégée
<code>/orders</code>	OrderHistory — Protégée
<code>/order-confirmation/:orderId</code>	OrderConfirmation — Protégée
<code>/wishlist</code>	Wishlist — Protégée

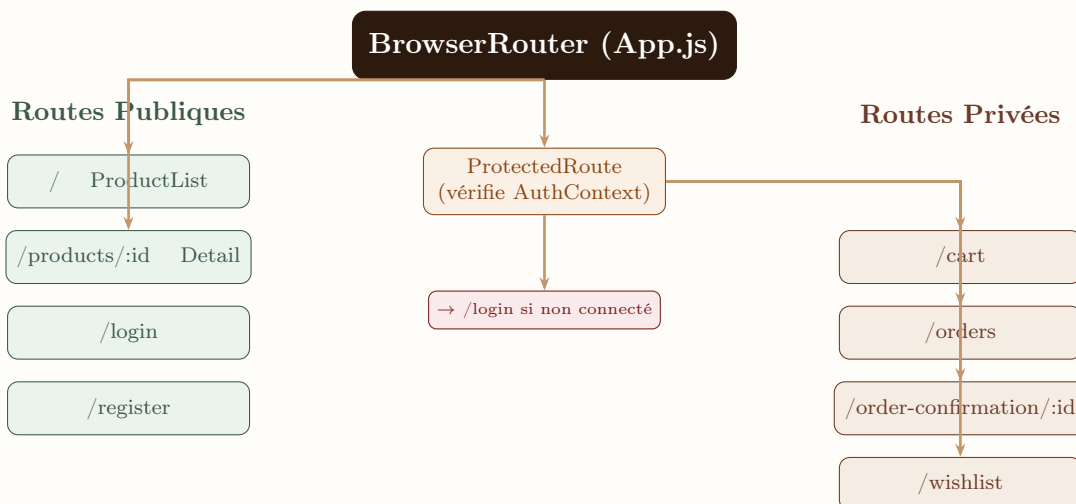


Figure 10.1: Arbre de routing React — routes publiques vs routes protégées par ProtecteRoute

10.2 Couche Service — api.js (Axios)

api.js centralise tous les appels HTTP. Instance Axios créée avec `axios.create({baseUrl: 'http://localhost:8080/api', headers: {'Content-Type': 'application/json'}})`. Un intercepteur request lit le user depuis `localStorage` pour permettre une future injection de token JWT (actuellement commenté). Chaque fonction exportée correspond à un endpoint précis : les composants React appellent `getProducts()` sans connaître l'URL réelle.

10.3 Contextes React

AuthContext

Gère la session utilisateur globale. `useState` pour `user` et `loading`. `useEffect` de montage lit `localStorage.getItem('user')` et `JSON.parse`. Méthodes :

- `login(email, password)` — appelle l'API, stocke dans state ET `localStorage`
- `register(userData)` — appelle l'API, sans connexion automatique
- `logout()` — efface state ET `localStorage`, redirige vers `/login`

CartContext

Gère l'état du panier synchronisé avec le backend. `useEffect` surveille `user` : connexion → `fetchCart()` ; déconnexion → `setCart(null)`. Toutes les mutations (`addItem`, `updateItem`, `removeItem`, `emptyCart`) appellent l'endpoint correspondant et mettent à jour le state local depuis la réponse serveur.

WishlistContext

Gère une liste de souhaits 100% côté client. Initialisée via `useState(() => JSON.parse(localStorage.getItem('wishlist') || '[]'))` (initialisation lazy). `useEffect` sur `wishlist` synchronise dans `localStorage` à chaque modification. Méthodes : `toggle(productId)`, `isWished(productId)`, `clearWishlist()`.

11 Composants Frontend — Détail Technique

11

Composants

Header, ProductList, Cart, OrderHistory...

11.1 Header et Navigation

Composant de navigation global, rendu en dehors du switch de routes pour être présent sur toutes les pages.

Effet scroll `useEffect` ajoute un listener `scroll` (passive) sur `window`. Si `scrollY > 12`, la classe CSS `'scrolled'` est ajoutée, déclenchant une ombre via CSS transition. Listener nettoyé dans la fonction de retour.

Recherche URL-driven Soumission du formulaire navigue vers `/products?search=terme` via `useNavigate` (partage de liens, bookmarking, navigation avant/arrière).

Dropdown profil `useRef` sur le wrapper, `mousedown` sur `document` ferme le menu au clic extérieur.

Badges temps réel Badge panier : `cart?.totalItems` depuis `CartContext` ; Badge wishlist : `count` depuis `WishlistContext`. Mis à jour automatiquement via `Context`.

11.2 ProductList — Catalogue et Filtres

Gère trois modes d'affichage via une logique de priorité dans `fetchProducts()` :

- Recherche : paramètre URL `?search=` non vide → `searchProducts(keyword)`
- Filtre catégorie : `selectedCat` non null → `getProductsByCategory(id)`
- Tous les produits : état par défaut → `getProducts()`

Tri côté client (`Array.sort`) en quatre modes : Featured (ordre serveur), Price Low/High, Top Rated. Skeleton Loading : 8 cartes fictives animées avec shimmer CSS et `animationDelay: i * 0.06s` pour un effet cascade.

11.3 ProductCard — Carte Produit

- Navigation via clic sur la carte ; `e.stopPropagation()` sur les boutons overlays
- Wishlist toggle : bouton cœur SVG overlay avec `useWishlist().toggle()`
- Quick add to cart : bouton + appelle `useCart().addItem(id, 1)`
- Out of stock overlay : si `availableQuantity <= 0`, overlay semi-transparent + badge, bouton d'ajout disabled
- Stock indicator : point coloré (vert > 5 , orange 1–5, rouge 0) avec texte dynamique

11.4 ProductDetail — Fiche Produit

- Sélecteur de quantité borné entre 1 et `product.availableQuantity`
- Feedback d'action : messages de succès/erreur disparaissant après 3200 ms via `setTimeout` nettoyé dans `useEffect`
- Bouton “Add to Cart” : reste sur la page avec message de confirmation
- Bouton “Buy Now” : ajoute au panier et navigue vers `/cart` via `useNavigate`
- États de chargement : boutons disabled avec texte “Adding...” pendant les requêtes

11.5 Cart — Panier et Checkout

`handlePlaceOrder()` appelle `createOrder(user.id)`. En cas de succès : `emptyCart()` sur `CartContext`, puis `navigate('/order-confirmation/' + orderId)`. En cas d'erreur : affichage de `response.data.message` — l'utilisateur reste sur la page du panier.

11.6 OrderHistory et OrderConfirmation

OrderHistory : récupère `getOrdersByUser(user.id)` au montage, triée par date décroissante côté serveur. Badges de statut colorés : CONFIRMED (vert), PENDING (orange), SHIPPED (bleu), DELIVERED (vert foncé), CANCELLED (rouge).

OrderConfirmation : récupère `getOrderById(orderId)` depuis les params de route. Affiche récapitulatif complet avec liens vers OrderHistory et catalogue.

12 Gestion Centralisée des Exceptions

12

Exceptions

GlobalExceptionHandler et structure des erreurs

La classe `GlobalExceptionHandler`, annotée `@RestControllerAdvice`, intercepte toutes les exceptions non gérées via AOP et les transforme en réponses JSON structurées. Aucun controller ne contient de logique de gestion d'erreur — ils délèguent toutes les exceptions.

12.1 Structure de la Réponse d'Erreur

Champ	Type et Description
<code>status</code>	Integer — Code HTTP numérique (ex : 404, 409, 400)
<code>message</code>	String — Message d'erreur contextualisé et lisible
<code>timestamp</code>	LocalDateTime — Moment exact de l'erreur (ISO-8601)
<code>errors</code>	Map<String, String> — Erreurs de validation champ par champ (uniquement pour les 400 avec <code>@Valid</code>)

12.2 Exceptions Gérées et Codes HTTP

Exception Java	Statut HTTP	Cas d'Usage Typique
<code>ResourceNotFoundException</code>	404 Not Found	Utilisateur, produit, commande introuvable par ID
<code>ResourceAlreadyExistsException</code>	409 Conflict	Email déjà enregistré lors de l'inscription
<code>RuntimeException</code>	400 Bad Request	Panier vide, stock insuffisant, mot de passe incorrect
<code>MethodArgumentNotValidException</code>	400 Bad Request	<code>@Valid</code> échoué — map des erreurs par champ DTO

Bonne Pratique Architecture

La centralisation dans `GlobalExceptionHandler` garantit une structure JSON cohérente pour toutes les erreurs de l'API. Le frontend peut parser uniformément : `response.data.message` pour les erreurs métier, `response.data.errors` pour les erreurs de validation champ par champ.

13 Configuration et Infrastructure

13

Configuration

application.properties, CORS et dépendances Maven

13.1 application.properties — Backend

Propriété	Valeur Configurée
spring.application.name	estore
Datasource URL	jdbc:mysql://localhost:3306/estore_db? useSSL=false&serverTimezone=UTC
spring.datasource .username	root
spring.datasource .password	"mot de passe sql" (à externaliser en variable d'env.)
spring.jpa.hibernate.ddl- auto	update
spring.jpa.show-sql	true
spring.jpa.properties .hibernate.format_sql	true
spring.jpa.properties .hibernate.dialect	org.hibernate.dialect.MySQL8Dialect
server.port	8080
logging.level.com.estore	DEBUG

13.2 Configuration CORS — CorsConfig.java

CorsConfig implémente WebMvcConfigurer (@Configuration). Règles CORS pour /api/** :

Paramètre CORS	Valeur Configurée
allowedOrigins	http://localhost:3000 uniquement
allowedMethods	GET, POST, PUT, DELETE, PATCH, OPTIONS
allowedHeaders	* (tous les headers)
allowCredentials	true
maxAge	3600 secondes (cache pre-flight 1 heure)

13.3 Dépendances Maven — pom.xml

Dépendance	Rôle
spring-boot-starter-parent 3.5.13	BOM parent — gestion centralisée des versions
spring-boot-starter-web	Spring MVC, Tomcat embarqué, Jackson (JSON)
spring-boot-starter-data-jpa	Spring Data JPA, Hibernate ORM, transactions
spring-boot-starter-validation	Hibernate Validator, @Valid, @NotBlank...
mysql-connector-j	Driver JDBC MySQL (scope runtime)
lombok	@Data, @NoArgsConstructor, @RequiredArgsConstructor...
spring-boot-devtools	Redémarrage à chaud en développement
spring-boot-starter-test	JUnit 5, Mockito, Spring Test (scope test)

14 Flux de Données End-to-End

14

Flux de Données

Inscription, panier et checkout pas à pas

14.1 Flux d'Inscription et de Connexion

Inscription

1. L'utilisateur remplit `Register.js` — validation locale (`password === confirmPassword`) puis `AuthContext.register()`
2. `registerAPI({firstName, lastName, email, password})` → `POST /api/auth/register`
3. `CustomerController.register()` reçoit `RegisterRequest` validé par `@Valid` — délègue à `CustomerService.register()`
4. Service vérifie l'unicité email, crée `User` + `Profile` vide, retourne `UserResponse`
5. HTTP 201 Created — `Register.js` navigue vers `/login`

Connexion

1. `AuthContext.login(email, password)` → `POST /api/auth/login`
2. Service : `findByEmail()` (404 si introuvable), compare mots de passe (400 si incorrect)
3. `AuthContext` stocke user dans state ET `localStorage`
4. `CartContext useEffect` détecte user non null → `fetchCart()` → `GET /api/cart/{userId}`
5. Header badges mis à jour automatiquement via `Context`

14.2 Flux d'Ajout au Panier

1. Clic sur le bouton + de `ProductCard` ou "Add to Cart" de `ProductDetail` → `CartContext.addItem(productId, quantity)`
2. `api.js` : `POST /api/cart/{userId}/add` avec body `{productId, quantity}`
3. `ShoppingService.addToCart()` : `getOrCreateCart()` → `inventoryService.checkAvailability()` [exception si insuffisant]
4. Produit déjà dans le panier ? Oui → incrémentation ; Non → nouveau `CartItem` avec `unitPrice` courant
5. `cartRepository.save()` → `CartResponse` → `CartContext` met à jour le state → badge du Header mis à jour

14.3 Flux de Passage en Caisse

1. L'utilisateur clique "Place Order" dans `Cart.js` → `createOrder(user.id)` → `POST /api/orders?userId={id}`
2. `BillingService.createOrder(userId)` démarre `@Transactional`
3. `findById(userId)` + `findByUserId(userId)` + vérification panier non vide
4. Boucle sur tous les `CartItems` : `inventoryService.checkAvailability()` — si un seul article en rupture → exception → rollback complet
5. Création `Order` (statut `CONFIRMED`), construction des `OrderItem` avec subtotals, calcul `totalAmount`
6. Boucle : `inventoryService.reserveStock()` décrémente `inventory.quantity` pour chaque article
7. `orderRepository.save(order)` avec cascade → tous les `OrderItems` persistés
8. `shoppingService.clearCart(userId)` vide le panier (`orphanRemoval` supprime les `CartItems`)
9. HTTP 201 + `OrderResponse` → `Cart.js` : `emptyCart()` + `navigate('/order-confirmation/' + orderId)`
10. `OrderConfirmation` : `GET /api/orders/{orderId}` → affichage récapitulatif

15 Analyse de Sécurité

15

Sécurité

Vulnérabilités, risques et points conformes

15.1 Vulnérabilités Critiques — Correction Requise en Production

CRITIQUE 1 — Mots de Passe Stockés en Clair

`CustomerService.register()` stocke `request.getPassword()` directement sans encodage. `CustomerService.login()` compare en texte clair. Solution : `BCryptPasswordEncoder` de Spring Security. Dans `register` : `user.setPassword(encoder.encode(request.getPassword()))`. Dans `login` : `encoder.matches(request.getPassword(), user.getPassword())`.

CRITIQUE 2 — Absence d'Authentification par Token

Aucun token JWT ni session côté serveur. L'`userId` est transmis en clair comme paramètre d'URL (`?userId=5`). N'importe qui peut forger une requête au nom d'un autre utilisateur. Solution : Spring Security + JWT (`io.jsonwebtoken:jjwt`), token en header `Authorization: Bearer <token>`.

CRITIQUE 3 — Absence de Contrôle d'Accès (RBAC)

Tous les endpoints API sont publics et non protégés. Aucun mécanisme n'empêche un utilisateur non authentifié d'accéder au panier ou aux commandes d'un autre utilisateur. Solution : `@PreAuthorize("#userId == authentication.principal.id")` sur chaque endpoint sensible.

15.2 Risques Modérés et Points Conformes

Aspect Sécurité	État Actuel et Recommandation
Rate Limiting	Absent — exposition aux attaques par force brute sur /api/auth/login. Solution : Bucket4j ou Resilience4j
HTTPS/TLS	Non configuré (env. dev). En production : certificat TLS + HSTS header
Secrets en config	Mot de passe MySQL en clair dans application.properties. Solution : variables d'environnement ou Vault
Injection SQL	Protégé — JPA utilise des requêtes paramétrées (PreparedStatement)
XSS	Partiellement protégé — React échappe automatiquement le contenu JSX
Exposition données	Correcte — UserResponse ne contient pas le champ password
Séparation Enti-ty/DTO	Correcte — aucune entité JPA exposée directement en JSON
Validation des entrées	Partielle — @Valid présent sur certains DTOs, couverture à compléter

16 Qualité du Code et Bonnes Pratiques

16

Qualité du Code

Lombok, transactions, DTOs et BigDecimal

16.1 Lombok — Réduction du Boilerplate

- `@Data` — génère getters, setters, `equals()`, `hashCode()`, `toString()`
- `@NoArgsConstructor` — constructeur sans arguments requis par JPA pour instancier les entités
- `@AllArgsConstructor` — constructeur avec tous les arguments (DTOs de réponse)
- `@RequiredArgsConstructor` — constructeur avec les champs `final` (injection par constructeur dans les services et controllers)

L'utilisation de `@RequiredArgsConstructor` avec `private final` est la méthode recommandée par Spring pour l'injection par constructeur, préférable à `@Autowired` sur champ (non testable sans Spring Context).

16.2 Transactions Spring `@Transactional`

Les méthodes de service modifiant des données portent `@Transactional`, garantissant les propriétés ACID : atomicité, cohérence, isolation et durabilité. La méthode `createOrder()` de `BillingService` est l'exemple le plus critique, orchestrant des opérations sur quatre repositories différents dans une seule transaction.

16.3 Pattern DTO — Découplage Entité/API

Aucune entité JPA n'est retournée directement par un endpoint REST. Ce pattern offre quatre avantages :

1. Évite l'exposition de champs sensibles (le champ `password` de `User` ne peut pas fuiter dans `UserResponse`)
2. Permet d'ajouter des champs calculés (`averageRating`, `availableQuantity` dans `ProductResponse`)
3. Préviens les `LazyInitializationException` lors de la sérialisation JSON des associations JPA lazy
4. Découple le modèle de données interne du contrat API public

16.4 BigDecimal pour les Valeurs Monétaires

Tous les prix et montants utilisent `java.math.BigDecimal` avec `@Column(precision=10, scale=2)`. `double` et `float` (IEEE 754) introduisent des erreurs d'arrondi imprévisibles ($0.1 + 0.2 \neq 0.3$ en flottant binaire). `BigDecimal` effectue des calculs décimaux exacts, indispensables pour la comptabilité.

16.5 Absence de Tests Automatisés

`EstoreApplicationTests` ne contient qu'une méthode `contextLoads()` auto-générée. Couvertures minimales recommandées en production :

- Tests unitaires des services avec Mockito (mock des repositories)
- Tests d'intégration des repositories avec `@DataJpaTest` et base H2 en mémoire
- Tests des controllers avec `@WebMvcTest` et `MockMvc`
- Tests end-to-end avec `@SpringBootTest` et TestContainers (MySQL réel dans Docker)

17 Expérience Utilisateur et Interface

17

Expérience Utilisateur

Design, états de chargement et accessibilité

17.1 Positionnement Design

L'application implémente un design system minimaliste à connotation premium, nommé "Artisan Atelier" dans le Header. Ce positionnement se retrouve dans les choix lexicaux ("Collection", "Curated Luxury"), dans les animations sobres (skeleton loading avec shimmer, transitions CSS délibérées) et dans la sobriété typographique.

17.2 États de Chargement

ProductList SkeletonGrid : 8 cartes fictives de taille identique aux cartes réelles, garantissant un layout stable (évite le CLS — Cumulative Layout Shift)

ProductDetail / Cart / OrderHistory Spinners centrés pendant le chargement initial

Boutons d'action États disabled avec texte "Adding..." ou "Placing Order..." pendant les requêtes asynchrones (évite les doubles clics)

17.3 États Vides

- ProductList sans résultats : illustration de panier vide, message "Nothing found", bouton "Clear filters"
- Cart vide : icône de panier, message "Your cart is empty", lien "Browse Collection" → `/products`
- OrderHistory sans commandes : message et bouton "Start Shopping" → `/products`

17.4 Accessibilité

- Boutons d'icônes munis d'attributs `aria-label` descriptifs
- ProductCard utilise `role='button'` et `onKeyDown` pour la navigation au clavier (touche Entrée)
- Toutes les images ont des attributs `alt` renseignés (nom du produit)
- Images de ProductCard chargées avec `loading='lazy'`

18 Feuille de Route et Améliorations

18

Feuille de Route

Améliorations prioritaires et vision à long terme

18.1 Priorité Haute — Sécurité (Pré-Production Obligatoire)

- Encodage BCrypt des mots de passe via PasswordEncoder de Spring Security dans CustomerService
- Intégration Spring Security avec chaîne de filtres de sécurité (SecurityFilterChain)
- Implémentation JWT (io.jsonwebtoken:jjwt) : génération à la connexion, validation dans OncePerRequestFilter
- @PreAuthorize sur les endpoints sensibles pour vérifier userId du token vs. URL
- Migration userId depuis paramètres d'URL vers claims JWT extraits du SecurityContext
- Externalisation des secrets (mot de passe DB) dans des variables d'environnement
- Couverture complète Bean Validation sur tous les DTOs de requête

18.2 Priorité Haute — Fonctionnalités Manquantes

- Interface de soumission d'avis clients : formulaire de notation (étoiles interactives) et commentaire sur ProductDetail
- Persistance serveur de la wishlist : entité Wishlist + WishlistItem, controller/service/repository dédiés
- Pagination du catalogue : @Pageable côté backend, interface de pagination ou infinite scroll côté React

18.3 Priorité Moyenne — Améliorations Fonctionnelles

- Panneau d'administration : gestion des produits (CRUD), catégories, stocks, tableau de bord des commandes
- Workflow de statut des commandes : transitions CONFIRMED → SHIPPED → DELIVERED avec timestamps
- Intégration passerelle de paiement : Stripe Checkout ou PayPal, webhook de confirmation
- Upload d'images produits : endpoint multipart/form-data, stockage S3 ou système de fichiers local
- Filtres avancés : plage de prix, note minimale, disponibilité (en stock uniquement)

- Moteur de recommandation simple : “Produits similaires” basé sur la catégorie

18.4 Priorité Basse — Mise en Production

- Docker Compose : services `mysql`, `spring-boot-api`, `nginx` (reverse proxy vers build React statique)
- Profils Spring Boot : `application-dev.properties`, `application-prod.properties`, `application-test.properties`
- Configuration HTTPS/TLS avec Let’s Encrypt (Certbot) ou certificat d’entreprise
- Pipeline CI/CD : GitHub Actions avec jobs build + test + lint + déploiement automatique
- Spring Boot Actuator → Prometheus → dashboard Grafana
- Logs structurés JSON avec Logback, agrégation dans stack ELK
- Cache Redis pour le catalogue produits (réduction requêtes MySQL sur les endpoints de lecture)
- CDN (CloudFront, Cloudflare) pour servir les images produits depuis des edge locations mondiales

19 Guide de Démarrage

19

Guide de Démarrage

Prérequis, installation et vérification

19.1 Prérequis

Outil	Version Minimale / Notes
Java Development Kit (JDK)	25 — Amazon Corretto 25 ou Eclipse Temurin recommandés
Apache Maven	3.8+ — ou utiliser le wrapper <code>mvnw</code> inclus (aucune installation requise)
MySQL Server	8.0+ — créer la base <code>estore_db</code> manuellement avant le premier démarrage
Node.js	18 LTS ou supérieur — requis pour npm et Create React App
npm	9+ — inclus avec Node.js
IDE Backend recommandé	IntelliJ IDEA avec plugin Lombok activé
IDE Frontend recommandé	VS Code avec extensions ES7+ React/Redux Snippets et Prettier

19.2 Démarrage du Backend

1. Créer la base de données MySQL :

```
CREATE DATABASE estore_db CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

2. Vérifier les identifiants dans `back-end/estore/estore/src/main/resources/application.properties`
3. Depuis `back-end/estore/estore/`, exécuter :

```
./mvnw spring-boot:run          # Linux / macOS
mvnw.cmd spring-boot:run        # Windows
```

4. Au premier démarrage : Hibernate crée toutes les tables, DataInitializer insère les données

de test

5. Tester : `http://localhost:8080/api/products`

19.3 Démarrage du Frontend

1. Depuis `front-end/estore-frontend/fronend/` :

```
npm install          # premiere fois uniquement
npm start
```

2. L'application s'ouvre sur `http://localhost:3000`

3. Connexion de test : `john@example.com` / `password123`

19.4 Vérification du Bon Fonctionnement

- **Backend — Catalogue** : `GET` `http://localhost:8080/api/products` → liste JSON de 4 produits avec stock et notes
- **Backend — Auth** : `POST` `http://localhost:8080/api/auth/login` avec `{"email":"john@example.com","password":"password123"}`
- **Frontend — Catalogue** : la page `/products` doit afficher la grille de 4 produits chargée depuis l'API
- **Frontend — Panier** : se connecter, ajouter un produit → badge du header affiche 1
- **Frontend — Checkout** : passer une commande → naviguer vers `/order-confirmation/{id}` avec récapitulatif

20 Glossaire Technique

20

Glossaire

Définitions des termes clés du projet

Terme	Contexte	Définition
SPA	Frontend	Single Page Application — HTML chargé une fois, contenu mis à jour dynamiquement par JavaScript sans rechargement complet
REST API	Backend	Representational State Transfer — style d'architecture exploitant les méthodes HTTP (GET, POST, PUT, DELETE) et les codes de statut
JPA	Backend	Java Persistence API — spécification Java pour la gestion de la persistance objet-relationnel
Hibernate	Backend	Implémentation ORM de référence pour JPA. Traduit les opérations JPA en SQL, gère le cache de session, les relations lazy/eager
DTO	Backend	Data Transfer Object — objet POJO dédié au transport de données entre couches, sans logique métier ni dépendance JPA
CORS	Backend	Cross-Origin Resource Sharing — mécanisme HTTP autorisant ou interdisant les requêtes provenant d'une origine différente du serveur
BCrypt	Sécurité	Algorithme de hachage de mots de passe adaptatif avec salt aléatoire et coût configurable, résistant aux attaques par table arc-en-ciel
JWT	Sécurité	JSON Web Token — token signé numériquement transportant des claims (id, rôles) en format compact. Permet l'authentification stateless
@Transactional	Backend	Annotation Spring délimitant une unité de travail atomique (ACID). Tout échoue → rollback automatique complet
Context API	Frontend	Mécanisme React natif pour partager des données entre composants sans prop drilling, via Provider et useContext()
Skeleton Loading	Frontend	Technique UX : placeholders animés (shimmer) pendant le chargement réel, évitant les sauts de layout brusques (CLS)
Axios	Frontend	Bibliothèque JavaScript HTTP basée sur les Promises, supportant intercepteurs, transformation des données et annulation
localStorage	Frontend	API Web stockant des données clé-valeur persistantes dans le navigateur (~5–10 MB, visible par JavaScript)
BigDecimal	Backend	Type Java pour les calculs décimaux à précision arbitraire. Indispensable pour les montants financiers (évite les erreurs IEEE 754)
Cascade JPA	Backend	Propagation automatique des opérations JPA (persist, merge, remove) depuis une entité parente vers ses entités associées
DDL Auto	Backend	Comportement Hibernate pour le schéma : update (modifie sans supprimer), create-drop (recrée au démarrage)
orphanRemoval	Backend	Paramètre @OneToMany : si une entité enfant est retirée de la collection parent, elle est automatiquement supprimée de la BDD
@PrePersist	Backend	Callback JPA exécuté juste avant la première insertion. Utilisé pour initialiser les timestamps automatiquement
CommandLine Runner	Backend	Interface Spring Boot exécutée après le démarrage complet du contexte. Utilisé par DataInitializer pour insérer les données initiales
BOM	Backend	Bill of Materials — fichier pom.xml parent gérant les versions de toutes les dépendances. spring-boot-starter-parent est le BOM Spring Boot

21 Conclusion

21

Conclusion

Bilan technique et perspectives

eStore constitue une démonstration technique aboutie d’une application web e-commerce full-stack, mettant en œuvre l’intégration cohérente de technologies Java et JavaScript modernes. Le projet démontre avec succès la conception et l’implémentation d’une architecture logicielle multicouche, du modèle de données relationnel jusqu’à l’interface utilisateur interactive.

Sur le plan architectural, l’application respecte rigoureusement le principe de séparation des responsabilités (*Separation of Concerns*) avec une organisation par modules de domaine : **customer**, **catalog**, **inventory**, **shopping** et **billing**. Chaque module est autonome avec ses propres couches Controller, Service, Repository, Entity et DTO, facilitant la compréhension du code, sa maintenance et son extension future.

Le parcours utilisateur complet est implémenté et fonctionnel : navigation dans un catalogue de produits avec recherche textuelle et filtrage par catégorie, consultation de fiches produit enrichies avec stock en temps réel et notes agrégées, gestion d’un panier persistant avec validation d’inventaire à chaque opération, passage en caisse transactionnel avec réservation de stock atomique, et consultation de l’historique des commandes avec statuts colorés.

Côté frontend, l’application démontre une utilisation avancée de React : gestion d’état global avec Context API sans bibliothèque tierce, routing protégé avec React Router DOM 6, séparation nette entre la couche service HTTP et les composants UI, skeleton loading, états vides engageants, et navigation URL-driven pour la recherche.

Les lacunes sécuritaires identifiées — absence d’encodage des mots de passe, d’authentification JWT et de contrôle d’accès — sont des limitations volontaires de périmètre, clairement documentées dans le code source par des commentaires `TODO`. Ces points constituent une feuille de route naturelle vers une application production-ready, tirant parti de la base de code existante qui fournit déjà une fondation architecturale solide et cohérente.

Ce projet représente une base d’apprentissage exhaustive couvrant les concepts fondamentaux du développement web full-stack : conception relationnelle de base de données, API REST avec Spring Boot, ORM avec Hibernate/JPA, gestion d’état React, communication asynchrone HTTP, patterns de service transactionnel et gestion centralisée des erreurs.

— Fin du Rapport Technique eStore —